RANDOMIZED & STREAMING ALGORITHMS PROJECT

The BJKST Algorithm for Counting Distinct Elements in an Input Stream

A Complete Theoretical & Empirical Investigation of Zeroth Frequency Moment (F_0) Estimation

Core Project Objectives Covered:

1. Formulate the streaming problem
Define F0 as the number of distinct identifiers in a stream and motivate approximate counting using 60,000 cricket-delivery records.

2. Specify the hash-function requirements
Describe uniform hash outputs, geometric trailing-zero behaviour, pairwise independence, duplicate consistency, and collision control.

3. Explain the BJKST adaptive-sampling procedure
Detail the sampling level z, bounded buffer B, duplicate handling, overflow-based pruning, and the estimator F0_hat = |B| x 2^z.

4. Establish probabilistic guarantees and evaluate the method
Use Markov's and Chebyshev's inequalities, together with a union bound, to analyse estimation error and report verified dataset cardinalities.

Based on Bar-Yossef, Jayram, Kumar, Sivakumar, and Trevisan (RANDOM 2002)

SECTION 1 • PROBLEM SCENARIO

The Distinct Elements Problem (F_0 Moment) in Data Streams

Consider a massive data stream **S = ⟨a_1, a_2, …, a_m⟩** arriving sequentially, where each token a_t belongs to a large universe **U = {1, 2, …, n}**. We want to find the number of **distinct** elements seen so far:

> $$F_0(S) = |\{ u \in U : u \text{ appears at least once in } S \}|$$

Why Exact Deterministic Algorithms Fail at Scale

- **Naive Hash Table / Set:** Storing every unique element seen so far requires **Ω(F_0 · log n)** bits of RAM.
- **Naive Bit-Vector / Bitmap:** Allocating 1 bit per possible universe element requires **Ω(n)** bits (for 64-bit IDs, n = 2^{64}).
- **Information-Theoretic Impossibility (Alon–Matias–Szegedy, 1996):** Any one-pass algorithm that computes F_0 *exactly* OR is *deterministic* MUST use **Ω(min(n, m))** bits of space.

> **The Streaming Goal:** Achieve sublinear, logarithmic space **O($(1/\varepsilon^2)$ · log n · log(1/δ))** by allowing both **randomization** (success probability ≥ 1 − δ) and **approximation** (estimate within $(1 \pm \varepsilon)F_0$).

SECTION 1 • EMPIRICAL DOMAIN SCENARIO

Case Study: Live Telemetry Stream (`cricket_deliveries.csv`)

Our project dataset contains **m = 60,000 ball-by-ball cricket delivery events**. As balls are bowled in real time across leagues, analytics engines must count distinct entities on the fly without retaining historical logs.

Streaming Query Target (Column / Composite Key)	Sample Stream Token	Total Arrivals (m)	Exact Distinct (F_0)	Exact Set Growth
Batting Team (`batting_team`)	`Team_E`	60,000	10	Tiny (O(1))
Stadium Venue (`venue`)	`Ground_24`	60,000	30	Small
Active Bowler (`bowler`)	`Bowler_140`	60,000	150	Moderate
Active Batter (`batter`)	`Batter_036`	60,000	200	Moderate
Batter–Bowler Matchup Pair	`(Batter_036, Bowler_140)`	60,000	28,899	Combinatorial Surge
Matchup + Venue + Runs Tuple	`(Batter_036, Bowler_140, Ground_24, 1)`	60,000	58,957	Linear in Stream (Ω(m))

Key Insight: While single attributes have small F_0, ad-hoc multi-attribute queries (e.g., unique batter–bowler–venue combinations) cause F_0 to explode to 58,957. BJKST bounds memory using a fixed-capacity sketch buffer regardless of F_0!

Statistical Properties of the Hash Function Blackbox

In BJKST, we **never inspect internal bit-shifts or modular arithmetic** of the hash function. We treat $\mathbf{h : U \to \{0,1\}^L}$ (where $M = 2^L \geq n^3$) as a blackbox drawn from a **2-wise independent family H** with three statistical guarantees:

Blackbox Property	Formal Mathematical Statement	Why BJKST Needs This Property
1. Marginal Uniformity	For all $u \in U$ and $y \in [M]$: **$\Pr[\, h(u) = y \,] = 1 / M = 2^{-L}$**	Ensures every bit of h(u) is an unbiased fair coin flip (Pr[bit = 0] = 1/2).
2. Geometric Trailing Zeros	Let `zeros(y)` = trailing 0s of binary y: **$\Pr[\, \text{zeros}(h(u)) \geq z \,] = 1 / 2^z = 2^{-z}$**	Acts as an idempotent subsampling sieve: duplicates always hash to the exact same `zeros(h(u))`, while 2^{-z} of distinct items survive level z.
3. Pairwise (2-Wise) Independence	For any distinct $u_1 \neq u_2$ and targets y_1, y_2: **$\Pr[\, h(u_1)=y_1 \wedge h(u_2)=y_2 \,] = 1 / M^2$**	Eliminates covariance between distinct items ($\text{Cov}(X_u, X_v) = 0$), so **$\text{Var}(\sum X_u) = \sum \text{Var}(X_u)$**.

Why Not a Fully Random Oracle? A truly random function requires Ω(n log n) bits to store its lookup table. A 2-wise independent hash blackbox (e.g., h(x) = (ax + b) mod p) needs only **2 random seed words = O(log n) bits**!

SECTION 3 • ALGORITHMIC INTUITION

From Single-Register Extremes to Adaptive Reservoir Subsampling

For each distinct item u in the stream, the blackbox assigns a trailing-zero count `zeros(h(u))`. At any threshold level **z**, the expected number of distinct stream items with `zeros(h(u)) ≥ z` is:

> $E[\text{ Number of distinct items surviving at level } z\ ] = F_0 / 2^z$

Why Tracking Only the Maximum Trailing Zeros (Flajolet–Martin) Is Unstable

- If we only record $\mathbf{R = \max_u zeros(h(u))}$, we are operating at the extreme tail where $\mathbf{F_0 / 2^R \approx 1}$.
- A single lucky hash value with many trailing zeros can double or quadruple 2^R, and estimates are restricted to powers of 2.

The BJKST Breakthrough: Keep a Sample Buffer of Size $k = \Theta(1/\varepsilon^2)$

- Instead of thinning the stream until only **1** item remains, BJKST lowers the threshold z so that roughly $\mathbf{k = \Theta(1/\varepsilon^2)}$ distinct items survive in a sample buffer **B**.
- Whenever **|B| > k**, BJKST increments $\mathbf{z \leftarrow z + 1}$ (halving the survival probability from 2^{-z} to $2^{-(z+1)}$) and evicts items in B with `zeros(h(u)) < z`.

SECTION 3 • ALGORITHM SPECIFICATION

The BJKST Technique: Step-by-Step Operational Rules

No programming syntax is needed—the entire BJKST algorithm is governed by three mathematical rules operating on a state tuple **(z, B)**, where **z ≥ 0** is an integer level and **B** is a set of capacity **k =** $\lceil c / \varepsilon^2 \rceil$:

1. **Rule 1 — Arrival & Trailing-Zero Evaluation:**
   When stream token a_t arrives, pass it to the hash blackbox to obtain **y = h(a_t)** and count its trailing binary zeros **r = zeros(y)**.
2. **Rule 2 — Geometric Sieve Filter:**
   If **r < z**, discard a_t immediately. If **r ≥ z**, insert the pair **(a_t, r)** into the buffer set **B** (if a_t is already present in **B**, set union leaves **|B|** unchanged).
3. **Rule 3 — Buffer Overflow & In-Place Halving:**
   While **|B| > k**:
   - Increment the subsampling threshold level: **z** ← **z + 1**.
   - Evict from **B** all stored pairs whose trailing zero count satisfies **r < z**.

> **Query Estimator at Stream End:** $\hat{F}_0 = |B| \times 2^z$

SECTION 4 • WORKED EXAMPLE SETUP

Example Setup Using `cricket_deliveries.csv`

Let us trace BJKST on the first **12 deliveries** of `cricket_deliveries.csv` to count distinct **bowlers**, setting the buffer capacity to **k = 3** (so any 4th distinct item in B triggers an overflow and level-up).

Input Stream S (12 balls, F_0 = 9 distinct bowlers, 3 duplicates):

⟨B140, B042, B138, B005, B003, B140(dup), B042(dup), B019, B088, B112, B005(dup), B077⟩

Distinct Bowler ID	Blackbox Hash h(u)	Trailing Zeros r = zeros(h(u))	Highest Level z Survived
Bowler_140 (B140)	1011**00**	**2**	Survives z = 0, 1, 2
Bowler_042 (B042)	0110**10**	**1**	Survives z = 0, 1 (Evicted at z = 2)
Bowler_138 (B138)	1001**11**	**0**	Survives z = 0 (Evicted at z = 1)
Bowler_005 (B005)	1110**01**	**0**	Survives z = 0 (Evicted at z = 1)
Bowler_003 (B003)	010**100**	**2**	Survives z = 0, 1, 2
Bowler_019 (B019), Bowler_077 (B077)	0101**10**, 1010**10**	**1**	Survives z = 0, 1 (Rejected/Evicted at z = 2)
Bowler_088 (B088), Bowler_112 (B112)	1101**01**, 001**100**	**0** and **2**	B088: z = 0 only; B112: z = 0, 1, 2

Complete BJKST Execution Trace (Capacity k = 3)

Ball	Token a_t	r = zeros	Filter r ≥ z?	Buffer B Contents (Bowler : r)	Level z	Action / Transition
1–3	B140, B042, B138	2, 1, 0	All ≥ 0 (Yes)	{(B140:2), (B042:1), (B138:0)}	0	Buffer reaches capacity |B| = 3
4	B005	**0**	**0 ≥ 0 (Yes)**	{(B140:2), (B042:1)} ***(after purge)***	**0 → 1**	**Overflow (|B|=4 > 3)! z←1; evict B138(0), B005(0)**
5	B003	2	2 ≥ 1 (Yes)	{(B140:2), (B042:1), (B003:2)}	1	Inserted; |B| = 3
6–7	B140, B042 (dups)	2, 1	≥ 1 (Yes)	{(B140:2), (B042:1), (B003:2)}	1	Already in B; duplicates ignored
8	B019	**1**	**1 ≥ 1 (Yes)**	{(B140:2), (B003:2)} ***(after purge)***	**1 → 2**	**Overflow (|B|=4 > 3)! z←2; evict B042(1), B019(1)**
9	B088	0	0 < 2 (No)	{(B140:2), (B003:2)}	2	Rejected immediately by filter
10	B112	2	2 ≥ 2 (Yes)	{(B140:2), (B003:2), (B112:2)}	2	Inserted; |B| = 3
11–12	B005(dup), B077	0, 1	< 2 (No)	{(B140:2), (B003:2), (B112:2)}	2	Rejected immediately by filter

Final Estimate: At end of stream, **$|B| = 3$** and **$z = 2$** ⇒ **$\hat{F}_0 = |B| \times 2^z = 3 \times 2^2 = 12$** (True $F_0 = 9$). Notice how B005 at Ball 11 was rejected without needing to remember that it was evicted at Ball 4!

How Markov's Inequality Bounds Buffer Overflow

> **Markov's Inequality:** For any non-negative random variable $Y \geq 0$ and threshold $\alpha > 0$, $\mathbf{Pr[\, Y \geq \alpha \,] \leq E[Y] / \alpha}$.

Step 1: Define the Survivor Count Random Variable Y_z

Fix any level **z**. For each distinct stream item **u** ∈ **D** (where |D| = F_0), define the indicator random variable $\mathbf{X_{u,z} = 1}$ if `zeros(h(u)) ≥ z`, and **0** otherwise. Since $\mathbf{E[X_{u,z}] = 2^{-z}}$, the total number of distinct items surviving at level z is:

> $Y_z = \sum_{u \in D} X_{u,z} \;\Rightarrow\; E[Y_z] = \sum_{u \in D} E[X_{u,z}] = F_0 / 2^z$ (by Linearity of Expectation)

Step 2: Apply Markov's Inequality to Prove z Does Not Overshoot

Let **z*** be the ideal target level where $\mathbf{k / 12 \leq E[Y_{z^*}] = F_0 / 2^{z^*} < k / 6}$. What is the probability that the buffer overflows capacity **k** at level **z*** (which would falsely force the algorithm to level up past z*)? Since $\mathbf{Y_{z^*} \geq 0}$, apply **Markov's inequality** with **α = k**:

> $Pr[\text{ Buffer overflows at level } z^* \,] = Pr[\, Y_{z^*} \geq k \,] \leq E[Y_{z^*}] / k < (k / 6) / k = \mathbf{1 / 6}$

Markov's Inequality on Squared Deviation (Chebyshev Bound)

To bound the probability that our final estimate $\hat{F}_0 = Y_z \cdot 2^z$ deviates from F_0 by more than εF_0, observe that:

$$| \hat{F}_0 - F_0 | \geq \varepsilon F_0 \;\Leftrightarrow\; | Y_z - E[Y_z] | \geq \varepsilon E[Y_z] \;\Leftrightarrow\; (Y_z - E[Y_z])^2 \geq \varepsilon^2 (E[Y_z])^2$$

Applying Markov's Inequality to $W = (Y_z - E[Y_z])^2 \geq 0$

- Applying Markov's inequality to the non-negative random variable $W = (Y_z - E[Y_z])^2$ yields Chebyshev's inequality:

  $\Pr[\, |Y_z - E[Y_z]| \geq \varepsilon E[Y_z] \,] \leq E[W] / (\varepsilon^2 (E[Y_z])^2) = \mathrm{Var}(Y_z) / (\varepsilon^2 (E[Y_z])^2)$.

- By **2-Wise Independence** of the hash blackbox, $\mathrm{Cov}(X_{u,z}, X_{v,z}) = 0$ for $u \neq v$, so:

  $\mathrm{Var}(Y_z) = \sum_{u \in D} \mathrm{Var}(X_{u,z}) = \sum_{u \in D} 2^{-z}(1 - 2^{-z}) \leq F_0 / 2^z = E[Y_z]$.

- Choosing buffer capacity $k = 72 / \varepsilon^2$ ensures $E[Y_{z^*}] \geq k / 12 = 6 / \varepsilon^2$, giving:

  $\Pr[\, |\hat{F}_0 - F_0| \geq \varepsilon F_0 \,] \leq 1 / (\varepsilon^2 E[Y_{z^*}]) \leq 1 / 6$.

Union Bound Summary: Total failure probability ≤ Pr[Overflow past z*] (1/6) + Pr[Deviation > εF_0] (1/6) = **1/3** (Success ≥ 2/3). Running O(log(1/δ)) parallel copies and taking their **median** boosts success to **1 − δ**!

SECTION 6 • EMPIRICAL EVALUATION & COMPLEXITY

Results on `cricket_deliveries.csv` & Complexity Comparison

Evaluating BJKST across all **60,000 deliveries** with buffer capacity **k = 512** demonstrates both exactness at small cardinalities and sub-1% relative error on massive composite streams:

Stream Entity in `cricket_deliveries.csv`	True F_0	Final Level z	Retained |B|	BJKST Estimate $\hat{F}_0$	Relative Error
Distinct Bowlers (`bowler`)	150	0	150	150	**0.00% (Exact)**
Distinct Batters (`batter`)	200	0	200	200	**0.00% (Exact)**
Unique Batter–Bowler Matchups	28,899	6 ($2^6 = 64$)	454	29,056	**+0.54%**
Unique (Batter, Bowler, Venue, Runs)	58,957	7 ($2^7 = 128$)	459	58,752	**−0.35%**

Space Complexity Advantage: Naive exact counting stores all 58,957 composite tuples ($\Omega(F_0 \log n)$ bits), whereas BJKST retains at most **k = 512 items** at any instant—an **99.1% reduction in active memory**!

SECTION 7 • CONCLUSION & KEY TAKEAWAYS

Summary of Project Contributions

1. Thangella Jeeven Tej

Problem formulation and dataset analysis: introduced the distinct-elements problem (F0), outlined the limitations of exact counting, and analysed the 60,000-row cricket dataset and its distinct counts.

2. Varshini Tumuluri

Hash functions and algorithmic foundations: explained uniformity, pairwise independence and trailing zeros, and presented adaptive subsampling, buffer management and the BJKST estimation rules.

3. Ch Bavesh Sai

Worked example and execution: demonstrated the illustrative 12-token stream, duplicate suppression, buffer overflow and sampling-level changes, and explained Markov's inequality for the overflow bound.

4. Guruvana Lakshmi Narayana

Probabilistic guarantees and evaluation: explained Chebyshev's inequality and the union bound, presented the verified dataset results, discussed space complexity, and summarised the project findings.

Thank You! Questions & Viva Discussion Welcome.